More Recursion

CSCI 1101

2024-04-18

Recursive Specifications

A **recursive specification** of a function or procedure is one in which the thing being specified is used within the specification.

For example, here is a recursive specification for counting down from a number:

- To count down from a positive number n, say the number n, then count down from the number $n - 1$.
- To count down from 0, just say, “go”.

It is often easy to turn a recursive specification into a recursive python function:

```
def countdown(n) :
  if n > 0 :
    print(f'{n}')
    countdown(n-1)
  else :
    print('go!')
```

Example: Recursive Palindrome

A *palindrome* is a string that is spelled the same backwards as forwards.

Here is a recursive specification for recognizing palindromes:

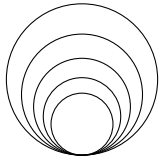
- The empty string is a palindrome.
- A nonempty string is a palindrome *if* its first and last letter are equal *and* the remaining letters, excluding the first and last, form a palindrome.

We can turn this recursive specification into a recursive python function:

```
def is_palindrome(text) :
  '''sig: (str) --> bool '''
  if text == '' :
    return True
  else :
    return text[0] == text[-1] and is_palindrome(text[1:-1])
```

Activity: Hawaiian Earring

A *Hawaiian earring* is a shape obtained by drawing a number of circles, all tangent at a point.



Here is a recursive specification for drawing a Hawaiian earring:

- To draw a Hawaiian earring with no hoops, do nothing.
- To draw a Hawaiian earring of size `s` with a positive number of hoops:
  - draw a circle of size `s`,
  - then draw a Hawaiian earring of size `4/5 * s` with one fewer hoops.

Turn this recursive specification into a recursive python function `earring(size , hoops)`.

Activity: Recursive List Reversal

Here is a recursive specification for reversing a list:

- The reversal of an empty list is an empty list.
- The reversal of a non-empty list consists of the reversal of its *tail* list, with its *head* element appended to the end.

Implement this recursive specification of list reversal as a recursive function

```
rev_list : (list a) --> list a
```

Example: Fibonacci Numbers

The **Fibonacci** function generates a sequence of numbers beginning with 0 and 1, and where each successive number is the sum of the previous two:

$$\text{fib}(n) \quad = \quad \begin{cases} n & \text{if } n = 0 \text{ or } n = 1 \\ \text{fib}(n-2) + \text{fib}(n-1) & \text{otherwise} \end{cases}$$

We can turn this recursive specification into a recursive python function:

```
def fib(n) :
  '''sig: (int) --> int '''
  if n in [0, 1] :
    return n
  else :
    return fib(n-2) + fib(n-1)
```

Activity: Greatest Common Divisor

The **greatest common divisor** ($\gcd$) of two positive integers m and n is the largest integer d that evenly divides both of them.

About 2300 years ago the Greek mathematician Euclid recorded a recursive algorithm to calculate the $\gcd$:

- If m evenly divides n then $\gcd(m, n) = m$.
- Otherwise, $\gcd(m, n) = \gcd(r, m)$, where r is the remainder of n divided by m.

Turn this ancient recursive algorithm into a recursive python function, `gcd(m , n)`.

Crinkles

A crinkle of depth 0 is just a straight line.
A crinkle of positive depth n is a four crinkles of depth $n-1$,
where the middle two bulge out to the side:

```
def crinkle(size , depth) :
  if depth <= 0 :
    turtle.forward(size)
  else :
    crinkle(size/3 , depth-1)
    turtle.right(60)
    crinkle(size/3 , depth-1)
    turtle.left(120)
    crinkle(size/3 , depth-1)
    turtle.right(60)
    crinkle(size/3 , depth-1)
```

Snowflakes

A **Koch snowflake** of depth n is an equilateral triangle, each side of which is formed by a crinkle of depth n.

```
def snowflake(size , depth) :
  for _ in range(3) :
    crinkle(size , depth)
    turtle.left(120)
```

To draw faster use `turtle.speed(0)` or `turtle.tracer(300 , 100)`

To Do

Finish *Project 3 - Automated Grader (Part 2)* on CodeRunner by midnight this Friday.

Finish *Homework 11: Recursion* on CodeRunner by 11:00 next Thursday.